MICROSAR SBC Driver

Technical Reference

Infineon Tlf35584

Version 1.00.00

Authors	Matthias Scheid
Status	Released

1 Document Information

1.1 History

Author	Date	Version	Remarks
Matthias Scheid	2017-11-30	1.00.00	Initial creation

1.2 Reference Documents

No.	Source	Title	Version
[1]	Infineon	TLF35584 Data Sheet	Rev. 2.0, 2017-03-16
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.2.1
[3]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	1.6.0
[4]	AUTOSAR	AUTOSAR_SWS_SPIHandlerDriver.pdf	3.2.0
[5]	AUTOSAR	AUTOSAR_SWS_WatchdogDriver.pdf	2.5.0

1.3 Scope of the Document

This technical reference describes the specific use of the SBC driver software. The SBC driver provides a well-defined API that is used by the periphery handlers (CAN transceiver driver, LIN transceiver driver and Watchdog driver) to control the SBC hardware and the corresponding components integrated on the System Basis Chip.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Document Information 2
1.1 History 2
1.2 Reference Documents 2
1.3 Scope of the Document 2

2 Hardware Overview 6

3 Introduction 7
3.1 Architecture Overview 7

4 Functional Description 9
4.1 Initialization 9
4.2 SBC States 10
4.2.1 UNINITIALIZED Mode 10
4.2.2 NORMAL Mode 10
4.2.3 STANDBY Mode 10
4.2.4 SLEEP Mode 11
4.3 Peripheral States 12
4.3.1 Watchdog states 12
4.3.1.1 OFF Mode 12
4.3.1.2 SLOW Mode 12
4.3.1.3 FAST Mode 12
4.4 Main Functions 12
4.5 Error Handling 12
4.5.1 Development Error Reporting 12

5 Integration 14
5.1 Scope of Delivery 14
5.1.1 Static Files 14
5.1.2 Dynamic Files 14
5.2 Critical Sections 15
5.3 Compiler Abstraction and Memory Mapping 17
5.4 Coordination with Watchdog driver 18
5.5 Mapping of SBC states and Device states 19
5.6 Watchdog configuration 19
5.6.1 Window watchdog (WWD) 19
5.6.1.1 Triggering via Dio 20
5.6.2 Functional Watchdog (FWD) 20
5.7 SPI Configuration 21

5.7.1 Device Configuration 21
5.7.2 Channel Configuration 21
5.7.3 Job Configuration 22
5.7.4 Sequence Configuration 22
5.7.5 SPI driver compatibility 22
5.8 Verification of hardware safety features 23
5.8.1 Inducing fault operation to window watchdog 23
5.8.2 Inducing fault operation to functional watchdog 23
5.9 Exclusive access to hardware registers 24

6 API Description 25
6.1 Type Definitions 25
6.2 Access macros for status flags 25
6.2.1 Macros for the system status 25
6.3 Event defines 26
6.3.1 Event defines for system events 26
6.4 Services provided by SBC 27
6.4.1 Sbc_30_Tlf35584_InitMemory 27
6.4.2 Sbc_30_Tlf35584_Init 27
6.4.3 Sbc_30_Tlf35584_GetVersionInfo 28
6.4.4 Sbc_30_Tlf35584_SetMode 28
6.4.5 Sbc_30_Tlf35584_GetStatus 29
6.4.6 Sbc_30_Tlf35584_ClearEvents 30
6.4.7 Sbc_30_Tlf35584_AccessRegister 30
6.4.8 Sbc_30_Tlf35584_Wdg_SetMode 31
6.4.9 Sbc_30_Tlf35584_Wdg_Trigger 31
6.5 Services used by SBC 32

7 Configuration 33
7.1 Configuration with DaVinci Configurator 5 33
7.1.1 Selection of the correct module definition 33

8 Limitations 34
8.1 Spi_DataBufferType 34
8.2 Unsupported Device states 34
8.3 No detection of Power-On-Resets 34

9 Abbreviations 35
9.1 Glossary 35
9.2 Abbreviations 35

10 Contact 36

Illustrations

Figure 3-1 AUTOSAR 4.x Architecture Overview 7
Figure 3-2 General concept of the SBC driver 8
Figure 4-1 Initialization order 9
Figure 4-2 SBC Modes 11
Figure 5-1 Configuration of SbcDioAccess 20
Figure 7-1 Module Definitions 33

Tables

Table 2-1 Supported Hardware Overview 6
Table 4-1 Service IDs 13
Table 4-2 Errors reported to DET 13
Table 5-1 Static files 14
Table 5-2 Generated files 14
Table 5-3 Compiler Abstraction and Memory Mapping 17
Table 5-4 Behavior defines for Watchdog driver 18
Table 5-5 Mapping of Sbc System modes and Device states 19
Table 5-6 SPI Device configuration 21
Table 5-7 SPI Channel Configuration 21
Table 5-8 SPI Job Configuration 22
Table 5-9 SPI Sequence Configuration 22
Table 5-10 Registers which require exclusive access 24
Table 6-1 Type definitions 25
Table 6-2 Macros for the system status 26
Table 6-3 Event defines for system events 26
Table 6-4 Sbc_30_Tlf35584_InitMemory 27
Table 6-5 Sbc_30_Tlf35584_Init 28
Table 6-6 Sbc_30_Tlf35584_GetVersionInfo 28
Table 6-7 Sbc_30_Tlf35584_SetMode 29
Table 6-8 Sbc_30_Tlf35584_GetStatus 29
Table 6-9 Sbc_30_Tlf35584_ClearEvents 30
Table 6-10 Sbc_30_Tlf35584_AccessRegister 31
Table 6-11 Sbc_30_Tlf35584_Wdg_SetMode 31
Table 6-12 Sbc_30_Tlf35584_Wdg_Trigger 32
Table 6-13 Services used by the SBC 32
Table 9-1 Glossary 35
Table 9-2 Abbreviations 35

2 Hardware Overview

The following table summarizes information about the SBC Driver. It gives you detailed information about the supported derivatives. In addition, the documentations of the hardware manufacturers are listed. The SBC driver bases on these documents in the given version.

Derivatives	Hardware Manual
TLF35584QVVS1 TLF35584QVVS2 TLF35584QKVS1 TLF35584QKVS2	TLF35584 Data Sheet, Rev. 2.0, 2017-03-16

Table 2-1 Supported Hardware Overview

3 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module SBC.

Supported AUTOSAR Release*:	4	
Supported Configuration Variants:	pre-compile	
Vendor ID:	SBC_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	SBC_MODULE_ID	255 decimal

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The MICROSAR SBC module provides an abstraction layer for an externally connected System Basis Chip. The module provides a hardware independent interface that can be used by upper layers such as CanTrcv, LinTrcv or Wdg to control the SBC hardware. The SBC module is implemented as Complex Device Driver (CDD) and as such uses types and memory- and compiler abstraction as defined in AUTOSAR.

3.1 Architecture Overview

The following figure shows where the SBC is located in the AUTOSAR architecture.

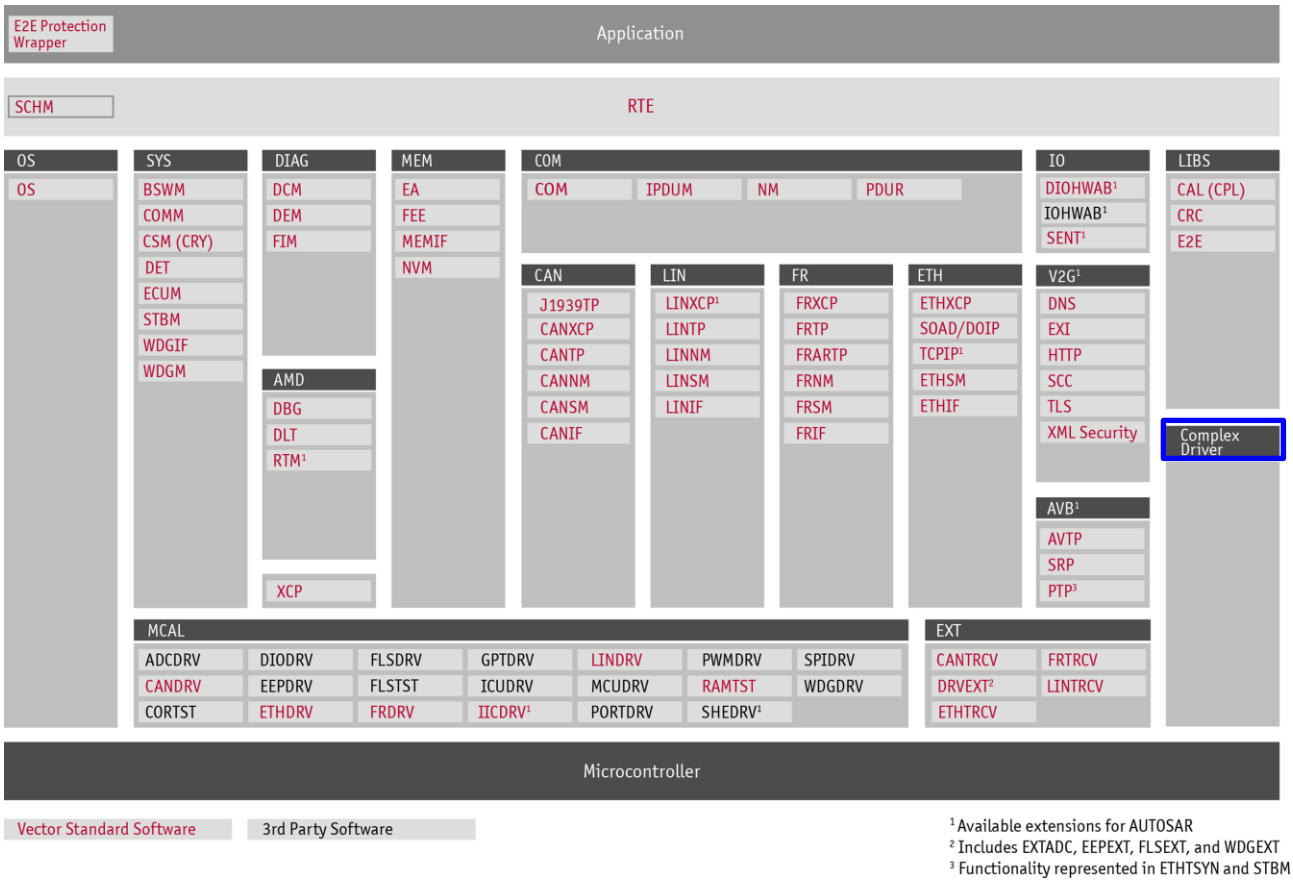


Figure 3-1 AUTOSAR 4.x Architecture Overview

A System Basis Chip contains various peripheries. To provide an AUTOSAR capable interface each supported periphery is wrapped by a generic AUTOSAR module such as CanTrcv, LinTrcv or Wdg driver. These periphery drivers provide a standardized AUTOSAR API so that a SBC module can easily be integrated into an AUTOSAR communication stack.
The generic AUTOSAR driver modules use a well-defined API that is independent from the underlying hardware to get access to the SBC hardware. The SBC driver provides that API and abstracts the SBC hardware access that is performed e.g. via an AUTOSAR SPI driver.
The next figure shows the general concept of the SBC driver.

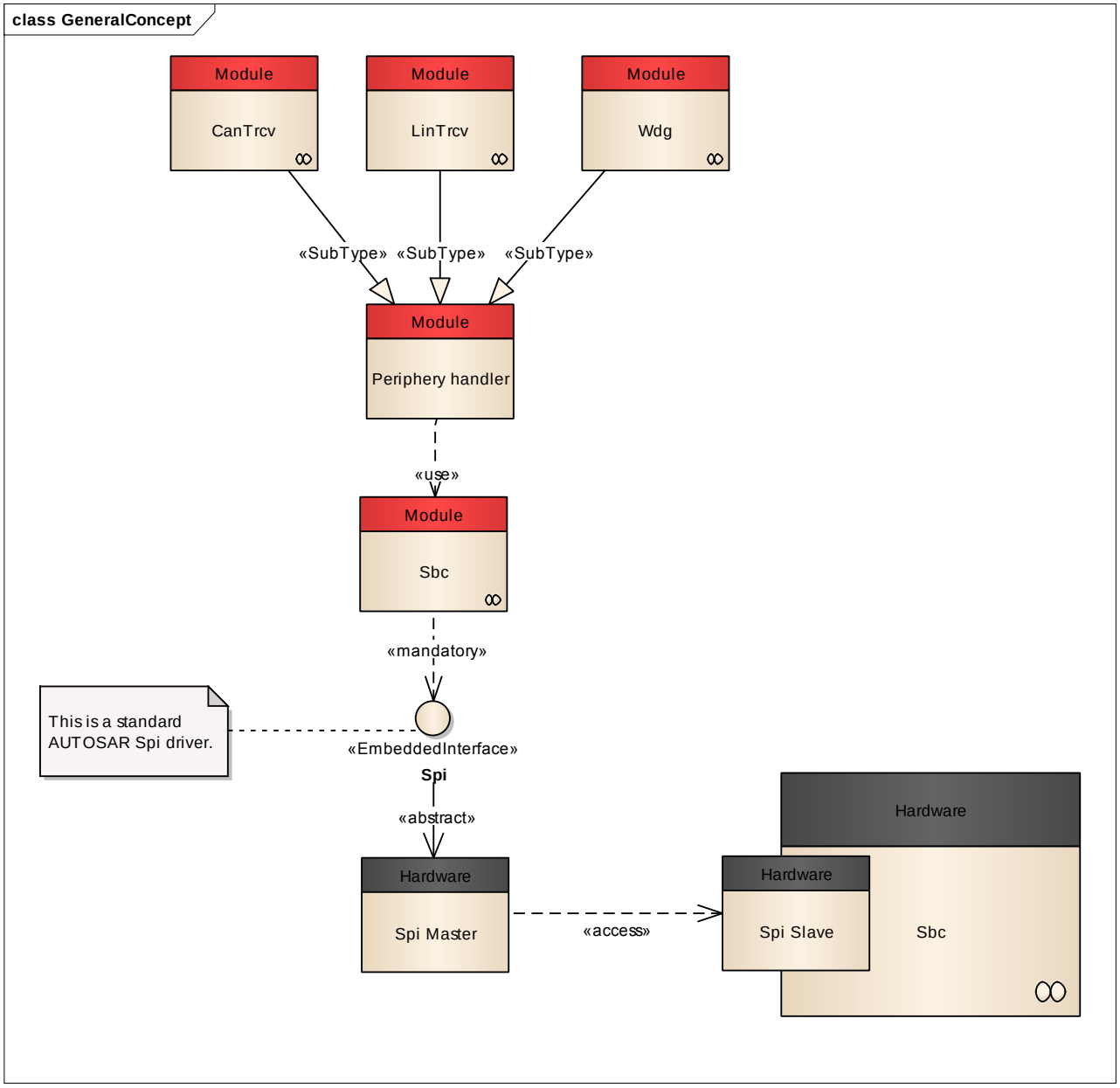


Figure 3-2 General concept of the SBC driver

4 Functional Description

4.1 Initialization

The SBC module is initialized at power on and after leaving the SLEEP Mode (see 4.2.4) by service function `Sbc_30_Tlf35584_Init()`. With this service the System Basis Chip is transferred into a safe state in which all configured peripherals are active. If the watchdog driver is not configured in generation tool the watchdog module on the SBC will be disabled by default. SBC initialization is only permitted at power-on and on MCU reset.

Caution
After power-on reset the service function `Sbc_30_Tlf35584_Init()` notifies this wake-up event (if configured) to EcuM component by calling the service function `EcuM_SetWakeupEvent()`. Therefore the optional wake-up source for SBC power-on reset must be configured in the EcuM module and referenced by SBC module in the generation tool.

The initialization sequence is strict and must be executed in the order like shown in Figure 4-1. The first step of the initialization sequence is to initialize the SBC driver. After that the peripheral driver (Wdg, CanTrcv, LinTrcv) are initialized. The initialization order of the different peripheral components is not strict.

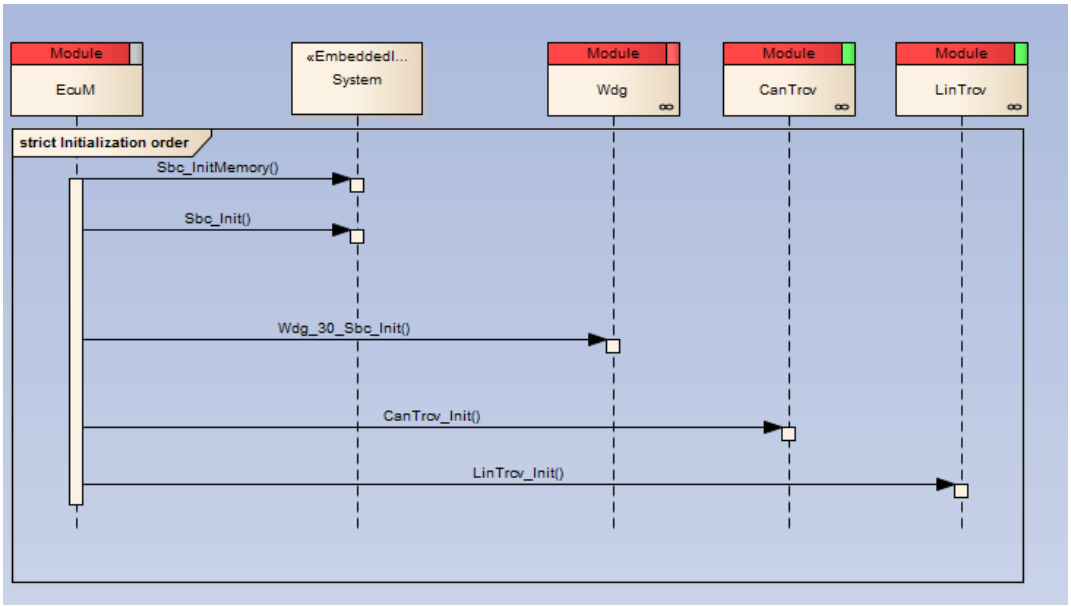


Figure 4-1 Initialization order

Caution
Depending on the watchdog hardware the watchdog timing window may start directly after the SBC hardware is switched from INIT to NORMAL Mode. So it is strongly recommended that the Wdg driver is initialized as soon as possible after Sbc driver is initialized.

4.2 SBC States

Figure 4-2 shows the different states and possible mode changes of the SBC module. Mode handling of the SBC modes must be performed by the application. See chapter 6 for detailed API description.

In the following the different states are described.

4.2.1 UNINITIALIZED Mode

The UNINITIALIZED mode is the mode of the SBC module before any initialization takes place. In this mode, the SBC hardware is in an undefined state. This mode can be left only by calling the service `Sbc_30_Tlf35584_Init()` which transfers the SBC module and corresponding hardware to the NORMAL mode.



Caution

When performing the initialization of the SBC module using service `Sbc_30_Tlf35584_Init()` and the module is unable to correctly communicate with the hardware the module mode remains UNINITIALIZED and the hardware remains in an undefined state. It is not possible to use the SBC module in this case.

As the SBC hardware is in an undefined state it may be necessary to trigger the hardware watchdog within a particular timeout. Hence it is necessary to call `Sbc_30_Tlf35584_Init()` as soon as possible. Refer to the hardware manual for further details about the Watchdog timeout during initialization.

4.2.2 NORMAL Mode

In NORMAL mode, all peripherals of the SBC are in a safe state. The watchdog must be triggered accordingly. Mode of individual peripherals can be changed without restrictions. The NORMAL mode can be entered from UNINITIALIZED or SLEEP mode via service `Sbc_30_Tlf35584_Init()` and from STANDBY mode by calling the service `Sbc_30_Tlf35584_SetMode()` with parameter SBC_SYS_MODE_NORMAL. The NORMAL mode can be left to STANDBY mode by calling the service `Sbc_30_Tlf35584_SetMode()` with the transfer parameter SBC_SYS_MODE_STANDBY. When SBC module is in NORMAL mode the MCU is supplied and communication is possible.

4.2.3 STANDBY Mode

In STANDBY mode the SBC is in a low power mode but is still supplying the MCU (i.e. important power supplies are still activated). All communication peripherals are deactivated (OFFLINE). It may be required to trigger the watchdog accordingly.

The STANDBY mode can be entered from NORMAL mode by calling the service `Sbc_30_Tlf35584_SetMode()` with parameter SBC_SYS_MODE_STANDBY and can be left by using this service with parameter SBC_SYS_MODE_NORMAL.

Caution
Entering STANDBY mode will not modify any peripheral mode. There may be constraints that prevent reaching the STANDBY state. In this case the `Sbc_30_Tlf35584_SetMode()` service returns E_NOT_OK and reports DET SBC_E_INVALID_STATE. In this case the SBC module remains in NORMAL mode.

4.2.4 SLEEP Mode

In SLEEP mode the SBC is in a low power mode and is no longer supplying the MCU (i.e. all power supplies are disabled). All communication peripherals are disabled and watchdog must not be triggered anymore.

The SLEEP mode can be entered from STANDBY mode by calling the `Sbc_30_Tlf35584_SetMode()` with parameter SBC_SYS_MODE_SLEEP and can be left only by service `Sbc_30_Tlf35584_Init()`.

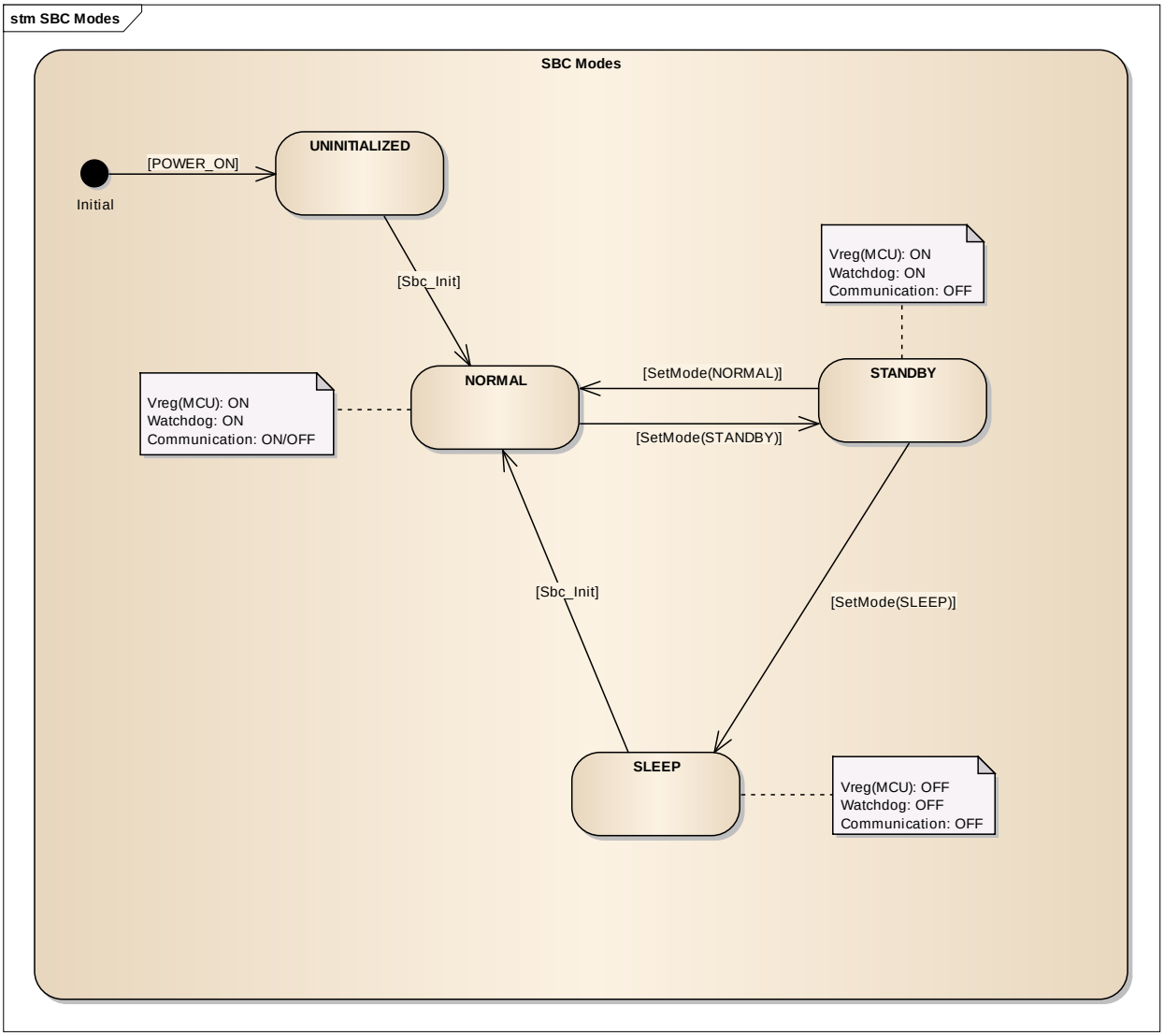


Figure 4-2 SBC Modes

4.3 Peripheral States

4.3.1 Watchdog states

For the watchdog module that is integrated into the SBC hardware all kind of state changes are permitted. Possible states are: OFF mode, SLOW mode and FAST mode like described in AUTOSAR specification [5].

Watchdog modes are switched by calling `Sbc_30_Tlf35584_Wdg_SetMode()`. See chapter 6 for detailed API description. The mode handling is performed by generic AUTOSAR watchdog driver.

4.3.1.1 OFF Mode

The watchdog hardware is disabled / shut down. Triggering the watchdog hardware is not necessary.



Caution
"This might be necessary in order to shut down the complete ECU and not get cyclic resets from a still running external watchdog. This mode might not be allowed for safety critical systems. In this case, the Wdg module has to be configured to prevent switching to this mode." [5]

4.3.1.2 SLOW Mode

The watchdog hardware is triggered with a long timeout period.

4.3.1.3 FAST Mode

The watchdog hardware is triggered with a short timeout period.

4.4 Main Functions

The SBC driver does not provide any main function.

4.5 Error Handling

4.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `SBC_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported SBC ID is 255.

The reported service IDs identify the services which are described in 6. The following table presents the service IDs and the related services:

Service ID	Service
0x00	SBC_SID_INIT
0x10	SBC_SID_GETVERSIONINFO
0x20	SBC_SID_SETMODE
0x21	SBC_SID_GETSTATUS
0x22	SBC_SID_CLEAREVENTS
0x24	SBC_SID_ACCESS_REGISTER
0x50	SBC_SID_WDG_SETMODE
0x51	SBC_SID_WDG_TRIGGER

Table 4-1 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	SBC_E_NO_ERROR: API service used to check if no error occurred.
0x09	SBC_E_PARAM_VERSIONINFO: API service used with an invalid versionInfo pointer.
0x0A	SBC_E_PARAM_ID: API service used with an invalid Sbc identifier.
0x0B	SBC_E_PARAM_MODE: API service used with an invalid mode.
0x0C	SBC_E_PARAM_STATUS: API service used with an invalid status.
0x0D	SBC_E_PARAM_SPIANSWER: API service used with an invalid SPI answer pointer.
0x10	SBC_E_UNINIT: API service used without module initialization.
0x11	SBC_E_ALREADY_INITIALIZED: The service `Sbc_30_Tlf35584_Init()` is called while the module is already initialized.
0x12	SBC_E_NULL_POINTER: API service called with NULL_PTR as parameter.
0x13	SBC_E_INVALID_STATE: API is called while the SBC is in an invalid state.
0x14	SBC_E_INITIALIZATION_FAILED: The initialization was not successful.
0x20	SBC_E_COMMUNICATION: No or invalid communication towards the hardware.
0x21	SBC_E_MODE_FAILED: Failed to set the peripheral mode.

Table 4-2 Errors reported to DET

5 Integration

This chapter gives necessary information for the integration of the MICROSAR SBC into an application environment of an ECU.

5.1 Scope of Delivery

The delivery of the SBC contains the files which are described in the sections 5.1.1 and 5.1.2:

5.1.1 Static Files

File Name	Description
Sbc_30_Tlf35584.c	This is the source file of the SBC driver. This file contains the implementation of a hardware specific SBC module.
Sbc_30_Tlf35584.h	This is the header file of the SBC driver. This file contains public definitions of this SBC driver instance.
Sbc_30_Tlf35584_Types.h	This file contains type and macro definitions of this SBC driver instances.
Sbc_30_Tlf35584_Compiler_Cfg.inc	This include file contains the compiler abstraction of this SBC driver instance.
Sbc_30_Tlf35584_MemMap.inc	This include file contains the memory abstraction of this SBC driver instance.

Table 5-1 Static files

5.1.2 Dynamic Files

The dynamic files are generated by the configuration tool.

File Name	Description
Sbc_30_Tlf35584_Lcfg.c	This is the generated configuration file of this SBC driver instance. This file contains configuration dependent tables.
Sbc_30_Tlf35584_Cfg.h	This is the generated header file of this SBC driver instance. This file contains configuration dependent defines, types and declarations.

Table 5-2 Generated files

5.2 Critical Sections

The SBC driver uses the following critical sections.

SBC_EXCLUSIVE_AREA_0

This section ensures consistency while modifying the System status.

- Protects: Sbc_SystemStatus
- Used in: Sbc_30_Tlf35584_SetMode, Sbc_30_Tlf35584_GetStatus, Sbc_30_Tlf35584_ClearEvents
- Exclude: Sbc_30_Tlf35584_SetMode, Sbc_30_Tlf35584_GetStatus, Sbc_30_Tlf35584_ClearEvents
- Duration: Used while updating the Sbc_SystemStatus - SHORT duration

SBC_EXCLUSIVE_AREA_1

This section ensures consistency while modifying the CanTrcv status.

- Protects: Sbc_CanTrcvStatus
- Used in: Sbc_30_Tlf35584_CanTrcv_ReadStatus, Sbc_30_Tlf35584_CanTrcv_ClearEvents, Sbc_30_Tlf35584_CanTrcv_SetMode
- Exclude: Sbc_30_Tlf35584_CanTrcv_ReadStatus, Sbc_30_Tlf35584_CanTrcv_ClearEvents, Sbc_30_Tlf35584_CanTrcv_SetMode
- Duration: Used while updating the Sbc_CanTrcvStatus - SHORT duration

SBC_EXCLUSIVE_AREA_2

This section ensures consistency while modifying the LinTrcv status.

- Protects: Sbc_LinTrcvStatus
- Used in: Sbc_30_Tlf35584_LinTrcv_ReadStatus, Sbc_30_Tlf35584_LinTrcv_ClearEvents, Sbc_30_Tlf35584_LinTrcv_SetMode
- Exclude: Sbc_30_Tlf35584_LinTrcv_ReadStatus, Sbc_30_Tlf35584_LinTrcv_ClearEvents, Sbc_30_Tlf35584_LinTrcv_SetMode
- Duration: Used while updating the Sbc_LinTrcvStatus - SHORT duration

SBC_EXCLUSIVE_AREA_3

This section ensures consistent hardware access.

- Protects: SPI data transfer
- Used in: Sbc_LI_RwRegister
- Exclude: Other calls to SPI module
- Duration: Call to SPI module - LONG duration

5.3 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions of the Sbc and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	SBC_30_TLF35584_CODE	SBC_30_TLF35584_VAR_NOINIT	SBC_30_TLF35584_VAR_INIT	SBC_30_TLF35584_CONST
SBC_30_TLF35584_START_SEC_CODE SBC_30_TLF35584_STOP_SEC_CODE	■			
SBC_30_TLF35584_START_SEC_CONST_UNSPECIFIED SBC_30_TLF35584_STOP_SEC_CONST_UNSPECIFIED				■
SBC_30_TLF35584_START_SEC_VAR_INIT_8BIT SBC_30_TLF35584_STOP_SEC_VAR_INIT_8BIT			■	
SBC_30_TLF35584_START_SEC_VAR_NOINIT_8BIT SBC_30_TLF35584_STOP_SEC_VAR_NOINIT_8BIT		■		
SBC_30_TLF35584_START_SEC_VAR_NOINIT_16BIT SBC_30_TLF35584_STOP_SEC_VAR_NOINIT_16BIT		■		
SBC_30_TLF35584_START_SEC_VAR_NOINIT_32BIT SBC_30_TLF35584_STOP_SEC_VAR_NOINIT_32BIT		■		
SBC_30_TLF35584_START_SEC_VAR_NOINIT_SAFE SBC_30_TLF35584_STOP_SEC_VAR_NOINIT_SAFE		■		

Table 5-3 Compiler Abstraction and Memory Mapping



Caution
For SafeBSW environments some Sbc variables must be linked to a memory section that is protected against unqualified software access. For that special case the following memory mapping section is used:

```
SBC_30_TLF35584_START_SEC_VAR_NOINIT_SAFE
SBC_30_TLF35584_STOP_SEC_VAR_NOINIT_SAFE
```

5.4 Coordination with Watchdog driver

Due to the fact that the Watchdog driver is responsible for timing aspects it is necessary to coordinate the behavior of the Watchdog driver with the special needs of the Watchdog hardware on the System Basis Chip. Therefore, the Sbc configuration file (Sbc_30_Tlf35584_Cfg.h) is used to provide a set of behavior defines that describe the characteristics of the Watchdog hardware.

In Table 5-4 the macros are described. The Watchdog driver that directly uses the Watchdog services of the Sbc must adapt the timing behavior to the special needs of the underlying SBC Watchdog hardware.

Macro Name	Description
SBC_30_TLF35584_WDG_OFF_MODE_ALLOWED (STD_ON / STD_OFF)	Specifies whether the Watchdog hardware supports Off Mode.
SBC_30_TLF35584_WDG_SLOW_MODE_ALLOWED (STD_ON / STD_OFF)	Specifies whether the Watchdog hardware supports Slow Mode.
SBC_30_TLF35584_WDG_MODE_CHANGE_ASYNC (STD_ON / STD_OFF)	Specifies whether the Watchdog hardware changes the Mode after Watchdog is triggered (asynchronous Mode change).
SBC_30_TLF35584_WDG_INITIAL_TRIGGER (STD_ON / STD_OFF)	Specifies whether the SBC driver needs an initial trigger from watchdog driver: e.g. when SBC needs a watchdog trigger to leave INIT Mode or to set watchdog default mode.

Table 5-4 Behavior defines for Watchdog driver

5.5 Mapping of SBC states and Device states

As described in 4.2.2 the SBC state *STANDBY* is a low-power mode in which the MCU is still powered. For this reason, the *STANDBY* mode is mapped to the device *NORMAL* state. The MCU is permanently powered, the other power supplies offered by the TLF35584 can be configured for this

The SBC state *SLEEP* which is a low-power mode in which the MCU is not supplied. This state is mapped to the device state *STANDBY* which can be left by wakeup events.

For further information regarding device states, see the device manual ([1] Chapter 11, State Machine)

Sbc System Mode / TLF35584 states	STANDBY[	NORMAL	SLEEP	Not supported
STANDY			■	
NORMAL	■	■		
SLEEP				■
WAKE				■

Table 5-5 Mapping of Sbc System modes and Device states



Note
The SBC driver does not support all Device states. For further information refer to chapter 8.2.

5.6 Watchdog configuration

Although the TLF35584 provides two independent watchdog peripherals (WWD/FWD) the SBC driver only allows the exclusive usage of one of the watchdog peripherals. It is not capable to operate both watchdog peripherals simultaneously.

The watchdog timing of both watchdog peripherals is depending on the watchdog cycle time.

5.6.1 Window watchdog (WWD)

The window watchdog has a Closed Window (CW) and an Open Window (OW). The duration of both windows can be configured as multiples of 50 watchdog cycles. According to this configuration, the trigger timing must be adjusted via the adaption of the trigger period (`SbcWatchdogTriggerOffset`) and trigger offset (`SbcWatchdogTriggerOffset`). The parameters must be adjusted so that the open window is hit properly.

Example
The *watchdog cycle time* is configured to 0.1ms. The *OW* and *CW* are both configured to value 6 which is equivalent to 350 *watchdog cycles*. Accordingly, both windows have a duration of 35ms (350 x 0.1ms).

For the above settings, a valid watchdog timing is a trigger period which is set to 70ms and a trigger offset which is ≥ 50%.

5.6.1.1 Triggering via Dio

If the window watchdog should be triggered by Dio (via the WDI pin) instead of SPI the optional `SbcDioAccess` has to be configured in DaVinci Configurator 5 (see Figure 5-1 Configuration of SbcDioAccess).

The WWD only recognizes rising edges on input pin WDI as a valid trigger signal. To trigger the WWD via DIO, the following conditions must be fulfilled:

- The initial physical level of the used Dio Channel needs to be `STD_HIGH` (e.g. by PORT configuration).
- The trigger frequency needs to be doubled (by adapting the Watchdog Trigger Period) to recover the physical level of the WDI pin. (The level of the Dio Channel is flipped to STD_LOW between to triggers to be prepared for the next trigger.)

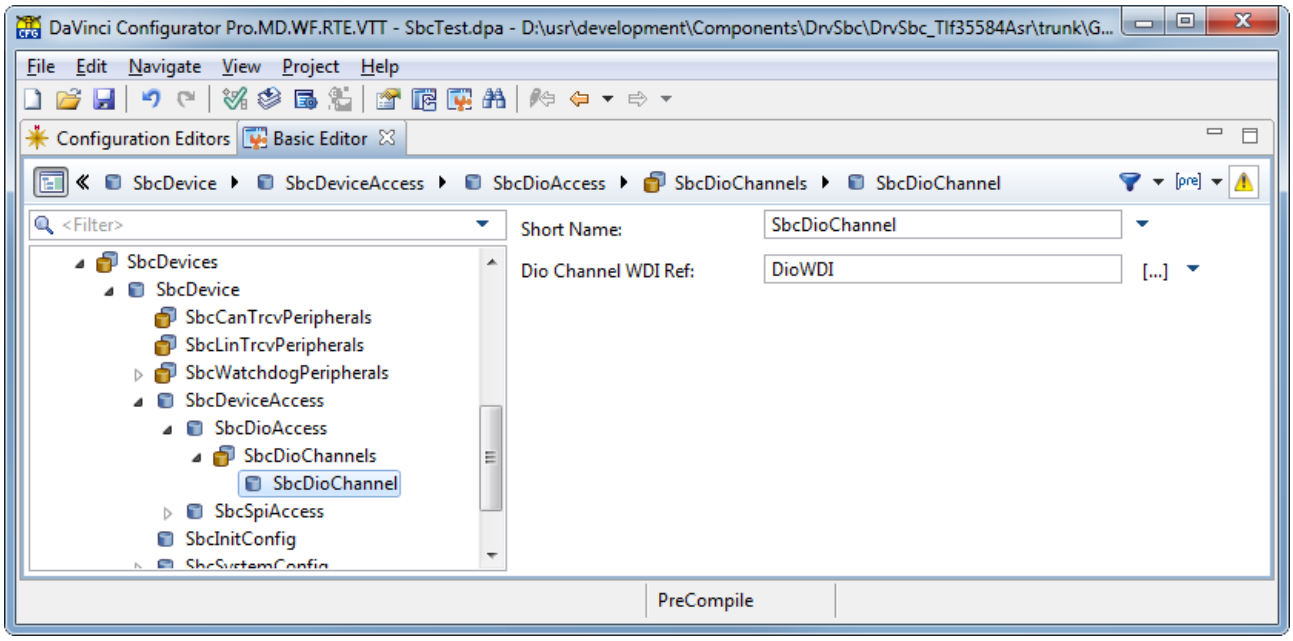


Figure 5-1 Configuration of SbcDioAccess

5.6.2 Functional Watchdog (FWD)

The functional watchdog has a functional heartbeat timer period which is configurable. The duration of this period is configured as a multiple of 50 watchdog cycles. According to this configuration, the trigger timing must be adjusted via the adaption of the trigger period (`SbcWatchdogTriggerOffset`) and trigger offset (`SbcWatchdogTriggerOffset`).

The parameters must be adjusted so that the heartbeat counter does not overflow (timeout).



Example
The *watchdog cycle time* is configured to 0.1ms. The *functional heartbeat timer period* is configured to value 11 which is equivalent to 600 *watchdog cycles*. Accordingly, the period has a duration of 60ms (600 x 0.1ms).

For the above settings a valid configuration is a trigger which is set to 60ms and a trigger offset which is small enough to ensure that the trigger is performed completely before the heartbeat timer overflows (e.g. ≤ 50%).

5.7 SPI Configuration

The SBC driver performs hardware access by calling service functions of the lower layer component SPI driver (see 6.5).

Therefore the SPI driver must be configured to allow access to the SBC driver. This means that device, sequences, jobs and channels must be defined prior using the SBC driver.

5.7.1 Device Configuration

Attribute	Value
Clock Idle Level	LOW (CPOL = 0)
Data Shift Edge	HIGH (CPHA = 1)
Chip Select Polarity	LOW

Table 5-6 SPI Device configuration

5.7.2 Channel Configuration

Attribute	Value
Channel Type	EB (External Buffer)
Data Width	16 Bit
EB Max Length	1
SPI Transfer Start	MSB

Table 5-7 SPI Channel Configuration

The channel has to be referenced in SBC driver configuration.

5.7.3 Job Configuration

Attribute	Value
Spi Hw Unit Synchronous	SYNCHRONOUS
Spi Job End Notification	NULL_PTR
Spi Job Priority	<Highest>
Assigned Device	Device configured in 5.7.1
Assigned Channels	Channel configured in 5.7.2

Table 5-8 SPI Job Configuration

5.7.4 Sequence Configuration

Attribute	Value
Assigned jobs	Job configured in 5.7.3

Table 5-9 SPI Sequence Configuration

The sequence has to be referenced in the SBC driver configuration.

5.7.5 SPI driver compatibility

For compatibility reasons, the Sbc driver provides the possibility to configure the interface parameter towards the Autosar SPI driver by inserting corresponding defines into the user configuration file of the SBC driver module.

The following four settings could be adapted to satisfy the needs of the used SPI driver.

1. *Type of application data buffer elements*

The SBC driver uses `Sbc_SpiDataType` as type of application data buffer elements for access to the elements of the external SPI buffers. The default for that type is `Spi_DataType` which is e.g. defined in the specification for ASR4.0.3, see [4].
If it is intended to use the driver with an ASR4.2.1 SPI driver, which provides the new type `Spi_DataBufferType`, the following code must be added to overwrite the default type:

```
#define Sbc_SpiDataType    Spi_DataBufferType
```

2. *Number of data elements to be transmitted*

The SBC driver uses `SBC_LL_SPI_TRANSFER_LENGTH` as value for the amount of data to be transmitted. The default value is 1 because the Sbc driver assumes that the SPI driver does transmit data elements of the SPI data width that is configured in 5.7.2.
If the SPI driver implementation uses that value to identify the amount of bytes to be transmitted, the following code must be added to overwrite the default value:

```
#define SBC_LL_SPI_TRANSFER_LENGTH    2
```

3. *Mapping of API to setup external buffers*

The SBC driver calls the Autosar SPI API `Spi_SetupEB` as default function to setup buffers and length of data specified for a special channel. If it is necessary for compatibility reasons to overwrite the function call, the following example code must be added:

```
#define Sbc_SpiSetupEB    Appl_SpiSetupEB
```

4. *Mapping of API to start synchronous transmission*

The SBC driver calls the Autosar SPI API `Spi_SyncTransmit` as default function to start synchronous data transmission for a special sequence. If it is necessary for compatibility reasons to overwrite the function call, the following example code must be added:

```
#define Sbc_SpiSyncTransmit   Appl_SpiSyncTransmit
```

5.8 Verification of hardware safety features

To verify the functionality of the TLF35584 watchdog safety features it might be necessary to intentionally induce fault operation.

5.8.1 Inducing fault operation to window watchdog

After successful initialization of the module fault operation of the Window Watchdog (WWD) can be provoked by triggering the window watchdog during the Closed Window. This can be achieved in different ways:

- Use the API `Sbc_30_Tlf35584_AccessRegister` to send a trigger to the Window watchdog service command register (*WWDSCMD*) during Closed Window.
- Use the API `Sbc_30_Tlf35584_Wdg_Trigger` to generate a trigger during Closed Window.
- Manipulate the level of the Dio Channel connected to WDI pin (in case Dio is used to trigger the WWD) to generate a trigger signal during Closed Window.

Since a valid trigger terminates the Open Window two consecutive triggers ensure that the Closed Window is hit.

5.8.2 Inducing fault operation to functional watchdog

After successful initialization of the module fault operation of the Functional Watchdog (FWD) can be provoked by using the API `Sbc_30_Tlf35584_AccessRegister` to send incorrect values to the *Functional watchdog response command* (*FWDRSP*) / *Functional watchdog response command with synchronization* register (*FWDRSPSYNC*).



Note
The Sbc driver does not provide any possibility to change the value of the *Window watchdog error threshold* (*WWDETHR)* or the *Functional watchdog error threshold* (*FWDETHR)*. If it is used in a SafeBSW environment it is also not possible to change the values via the API `Sbc_30_Tlf35584_AccessRegister` since any write access to watchdog configuration registers is denied.

5.9 Exclusive access to hardware registers

The registers listed in Table 5-10 are used exclusively by the Sbc driver and must not be altered by user software. Depending on the configuration (e.g. used watchdog peripheral) not all registers may be used.

Register	Description	Address
SYSPCFG0	Protected System configuration request 0	0x04
SYSPCFG1	Protected System configuration request 1	0x05
WDCFG0	Protected Watchdog configuration request 0	0x06
WDCFG1	Protected Watchdog configuration request 1	0x07
FWDCFG	Protected Functional watchdog configuration request	0x08
WWDCFG0	Protected Window watchdog configuration request	0x09
WWDCFG1	Protected Window watchdog configuration request	0x0A

Table 5-10 Registers which require exclusive access

6 API Description

6.1 Type Definitions

The types defined by the SBC driver are described in this section.

Type Name	C-Type	Description	Value Range
Sbc_IdType	uint16	Identifies a SBC instance. Encodes the SBC instance and a peripheral id.	`15..4: SBC device index 3..0: peripheral id`
Sbc_PeripheralIdType	uint16	Identifies a specific peripheral.	`15..4: SBC device index 3..0: peripheral id`
Sbc_SystemModeType	enum	System modes of the SBC.	`SBC_SYS_MODE_INVALID`
			`SBC_SYS_MODE_NORMAL`
			`SBC_SYS_MODE_STANDBY`
			`SBC_SYS_MODE_SLEEP`
Sbc_WdgModeType	enum	Wdg modes of the SBC.	`SBC_WDG_MODE_INVALID`
			`SBC_WDG_MODE_OFF`
			`SBC_WDG_MODE_SLOW`
			`SBC_WDG_MODE_FAST`
Sbc_SystemStatusType	struct	Stores the system status.	`Events, States, Mode`
Sbc_SystemEventType	uint16	Represents the events of a SBC instance.	All system events are integrated into that type.

Table 6-1 Type definitions

6.2 Access macros for status flags

The access macros that are defined by the SBC driver are described in this section. With these macros the SBC driver provides a comfortable way to interpret the status from the SBC and its peripheral components. The following macros are provided.

6.2.1 Macros for the system status

Macro Name	Description
Sbc_Sys_Ev_IsPor(systemStatus)	Returns whether the POR event is pending.
Sbc_Sys_Ev_IsTempFail(systemStatus)	Returns whether the TEMPFAIL event is pending.
Sbc_Sys_Ev_IsUverr(systemStatus)	Returns whether the UVERR event is pending.
Sbc_Sys_Ev_IsOverr(systemStatus)	Returns whether the OVERR event is pending.
Sbc_Sys_Ev_IsSpifail(systemStatus)	Returns whether the SPIFAIL event is pending.
Sbc_Sys_Ev_IsWdfail(systemStatus)	Returns whether the WDFAIL event is pending.
Sbc_Sys_Ev_IsOcerr(systemStatus)	Returns whether the OCERR event is pending.
Sbc_Sys_St_GetMode(systemStatus)	Returns the current mode.

Table 6-2 Macros for the system status



Example
Here is an example how to get the current system mode of the SBC.

```
Sbc_SystemStatusType systemStatus;
Sbc_GetStatus(id, &systemStatus);
if (Sbc_Sys_St_GetMode(systemStatus) == SBC_SYS_MODE_NORMAL ) {
  // do something
}
```

6.3 Event defines

The event defines that are provided by the SBC driver are described in this section. An example for their usage can be looked up in 6.2.1.

6.3.1 Event defines for system events

Macro Name	Description
SBC_SYS_EV_POR	Power-on reset occurred.
SBC_SYS_EV_TEMPFAIL	An over-temperature error occurred.
SBC_SYS_EV_UVERR	An under-voltage error occurred.
SBC_SYS_EV_OVERR	An over-voltage error occurred.
SBC_SYS_EV_SPIFAIL	A SPI failure occurred.
SBC_SYS_EV_WDFAIL	A watchdog failure occurred.
SBC_SYS_EV_OCERR	An over-current error occurred.

Table 6-3 Event defines for system events

6.4 Services provided by SBC

6.4.1 Sbc_30_Tlf35584_InitMemory

Prototype	
`void Sbc_30_Tlf35584_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Function for *_INIT_*-variable initialization. Service to initialize module global variables at power up. This function can be used to initialize the variables in *_INIT_* sections in case they are not initialized by the startup code.	
Particularities and Limitations	
Module must not be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-4 Sbc_30_Tlf35584_InitMemory

6.4.2 Sbc_30_Tlf35584_Init

Prototype	
`void Sbc_30_Tlf35584_Init (const Sbc_30_Tlf35584_ConfigType *ConfigPtr)`	
Parameter	
ConfigPtr [in]	Configuration structure for initializing the module.
Return code	
void	none
Functional Description	
Initialization function. The API initializes each configured device. For each device all corresponding status variables are set to an initialized state. The API then initializes the underlying hardware device and sets all embedded peripherals to a safe state, e.g. a state that does not influence any attached device or bus. If the underlying hardware device has detected a Power-on-Reset (POR) event, the API reports the event to the EcuM by calling EcuM_SetWakeupEvent with the corresponding configured wake-up source. If any error occurs during the initialization the module remains in an uninitialized state.	
Particularities and Limitations	
This API must not be interrupted by any other API of the SBC module. Interrupts have to be disabled. The module has to be uninitialized. The function Sbc_InitMemory must be	

called before the function Sbc_Init can be called. This function must be called before any other service functionality of the Sbc driver. The SPI driver must be initialized.
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 6-5 Sbc_30_Tlf35584_Init

6.4.3 Sbc_30_Tlf35584_GetVersionInfo

Prototype	
`void Sbc_30_Tlf35584_GetVersionInfo (Std_VersionInfoType *versioninfo)`	
Parameter	
versioninfo [out]	Pointer to where to store the version information
Return code	
void	none
Functional Description	
Returns the version information. This API returns version information, vendor ID and AUTOSAR module ID of the component. The versions are decimal-coded.	
Particularities and Limitations	
Output parameter must not be NULL.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-6 Sbc_30_Tlf35584_GetVersionInfo

6.4.4 Sbc_30_Tlf35584_SetMode

Prototype	
`Std_ReturnType Sbc_30_Tlf35584_SetMode (Sbc_IdType id, Sbc_SystemModeType mode)`	
Parameter	
id [in]	Identifies the SBC instance that shall change its mode.
mode [in]	Specifies the mode that the SBC instance shall be switched to.
Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.

Functional Description
Changes the mode of the SBC module. This API changes the system mode of the SBC identified by SBC id to the given mode.
Particularities and Limitations
When the mode of the hardware could not be changed successfully the internal (software) mode remains unchanged and the mode of the hardware is undefined. In order to query the actual operation mode of the underlying hardware the API Sbc_GetStatus() must be used. The module must be initialized.
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-7 Sbc_30_Tlf35584_SetMode

6.4.5 Sbc_30_Tlf35584_GetStatus

Prototype	
`Std_ReturnType` **`Sbc_30_Tlf35584_GetStatus`** `(Sbc_IdType id, Sbc_SystemStatusType *status)`	
Parameter	
id [in]	Identifies the SBC instance that shall be queried.
status [out]	Pointer to the object where the API should store the current status, pointer must be valid.
Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.
Functional Description	
Reads the current system status of the SBC. This API queries the current state from the underlying hardware and writes it to the object that is referred by parameter status. Note: If E_NOT_OK is returned the value of status is undefined.	
Particularities and Limitations	
The module must be initialized. Output parameter must not be NULL.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-8 Sbc_30_Tlf35584_GetStatus

6.4.6 Sbc_30_Tlf35584_ClearEvents

Prototype	
`Std_ReturnType` **`Sbc_30_Tlf35584_ClearEvents`** `(Sbc_IdType id, Sbc_SystemEventType events)`	
Parameter	
id [in]	Identifies the SBC instance whose events should be cleared.
events [in]	The events that should be cleared.
Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.
Functional Description	
Clears pending events from the SBC. This API clears the given events from SBC hardware. Events are cleared by writing "1" to them.	
Particularities and Limitations	
Only flags that are set in the given parameter "events" are cleared. The module must be initialized.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-9 Sbc_30_Tlf35584_ClearEvents

6.4.7 Sbc_30_Tlf35584_AccessRegister

Prototype	
`Std_ReturnType` **`Sbc_30_Tlf35584_AccessRegister`** `(Sbc_IdType id, Sbc_SpiDataType spiCommand, Sbc_SpiDataType *spiAnswer)`	
Parameter	
id [in]	Identifies the SBC instance whose register shall be accessed.
spiCommand [in]	The complete SPI value that can be directly written to MOSI register.
spiAnswer [out]	The complete SPI value that is received on MISO register . SpiAnswer must be a valid pointer.
Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.
Functional Description	
Access of SBC registers. The API allows the application to get access to specific SBC registers. Please refer to manual of semiconductor manufacturer to get knowledge about the structure of commands.	
Particularities and Limitations	
Please note that it is optional to include security or parity bits in the SPI command. They are recomputed	

and inserted by the Sbc driver itself. Be careful in usage of this API because manual access to SBC register can influence the behaviour of the SBC driver in an unwanted manner. The module must be initialized.
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-10 Sbc_30_Tlf35584_AccessRegister

6.4.8 Sbc_30_Tlf35584_Wdg_SetMode

Prototype	
`Std_ReturnType` **`Sbc_30_Tlf35584_Wdg_SetMode`** `(Sbc_PeripheralIdType id, Sbc_WdgModeType mode)`	
Parameter	
id [in]	Identifies the SBC instance and peripheral that shall change its mode.
mode [in]	The desired mode.
Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.
Functional Description	
Changes the mode of the watchdog to the given mode. The API changes the operation mode of the watchdog to either OFF, SLOW or FAST. The settings of each mode depend on hardware and configuration. If the hardware successfully changed the mode the new mode is stored in the internal status variable.	
Particularities and Limitations	
When the mode of the hardware could not be changed successfully the internal (software) mode remains unchanged and the mode of the hardware is undefined. The module must be initialized. Configuration Variant(s): SBC_30_TLF35584_ENABLE_WDG	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-11 Sbc_30_Tlf35584_Wdg_SetMode

6.4.9 Sbc_30_Tlf35584_Wdg_Trigger

Prototype	
`Std_ReturnType` **`Sbc_30_Tlf35584_Wdg_Trigger`** `(Sbc_PeripheralIdType id)`	
Parameter	
id [in]	Identifies the SBC instance and peripheral that shall be triggered.

Return code	
Std_ReturnType	E_OK on success. E_NOT_OK otherwise.
Functional Description	
Triggers the watchdog peripheral. The API triggers the hardware watchdog. The API has to be called according to the timing requirements of the selected watchdog mode.	
Particularities and Limitations	
Must not be interrupted by any other WDG call. The module must be initialized. Configuration Variant(s): SBC_30_TLF35584_ENABLE_WDG	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-12 Sbc_30_Tlf35584_Wdg_Trigger

6.5 Services used by SBC

In the following table services provided by other components, which are used by the SBC are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
ECUM	EcuM_SetWakeupEvent
SPI	Spi_SetupEB
SPI	Spi_SyncTransmit
DIO	Dio_FlipChannel

Table 6-13 Services used by the SBC

7 Configuration

7.1 Configuration with DaVinci Configurator 5

Refer to the integrated online help and parameter descriptions of Configurator 5.

7.1.1 Selection of the correct module definition

In Figure 7-1 an example of a module definition list is shown. There may be different SBC module definitions available in the Module Assistant. The definition with the Definition Reference /MICROSAR/Sbc/Sbc is the base description for the module. Please choose the corresponding platform specific definition (e.g. /MICROSAR/Sbc_Mc33908/Sbc).

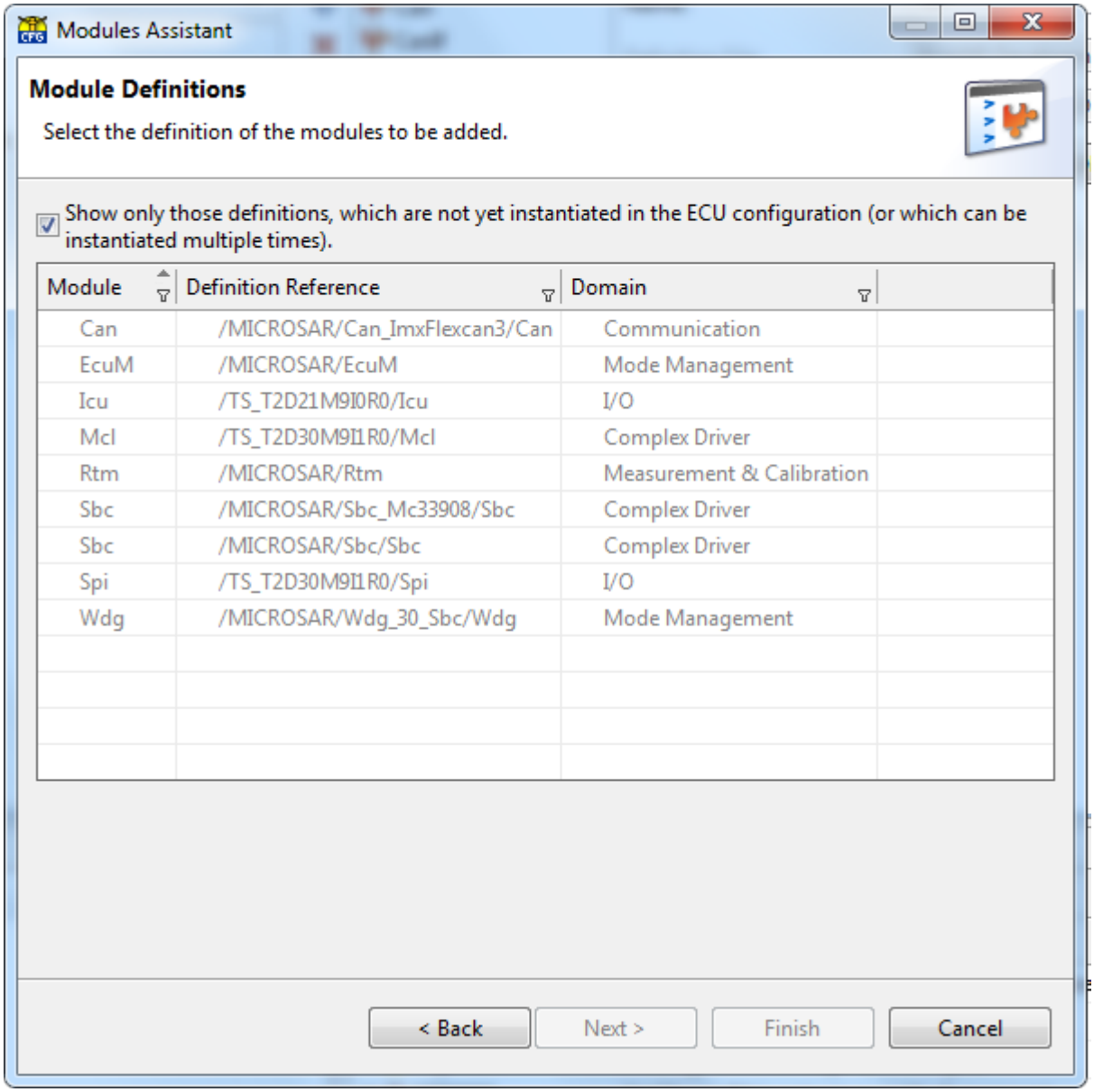


Figure 7-1 Module Definitions

8 Limitations

8.1 Spi_DataBufferType

The SBC driver uses the type `Spi_DataType` which is defined in the specification for ASR4.0.3, see [4].
If it is intended to use the driver with an ASR4.2.1 SPI driver, which provides the new type `Spi_DataBufferType`, the following code must be inserted into user configuration file of the SBC driver module:

```
#define Sbc_SpiDataType    Spi_DataBufferType
```

8.2 Unsupported Device states

The device states *WAKE* and *SLEEP* are not supported by the SBC driver. If the device should be set to these states this can be achieved manually via the API Sbc_30_Tlf35584_AccessRegister.

8.3 No detection of Power-On-Resets

Due to hardware limitations, it is not possible to detect Power-On-Reset events.

9 Abbreviations

9.1 Glossary

Term	Description
Cfg5	DaVinci Configurator 5

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CAN	Controller Area Network
CanTrcv	CAN transceiver
CDD	Complex Device Driver
CW	Closed Window
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DIO	Digital Input Output
ECU	Electronic Control Unit
ECUM/EcuM	ECU State Manager
FWD	Functional watchdog
ISR	Interrupt Service Routine
LIN	Local Interconnect Network
LinTrcv	LIN transceiver
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
OW	Open Window
SBC	System Basis Chip
SPI	Serial Peripheral Interface
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification
WDG/Wdg	Watchdog
WWD	Window Watchdog

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com